

[1] Operating System Introduction and Functions

- ⇒ It is a system software
- ⇒ It is an interface between user and hardware.
- ⇒ Resource Allocator
- ⇒ Manager :
 - Memory
 - Process
 - Files
 - Security etc.

Goal :-

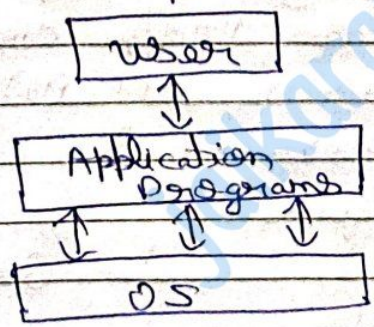
- ⇒ Primary Goal :- 1. Convenience of use (For small PCs) (Windows)
- 2. Throughput (Linux has good throughput)

Ex Windows (Microsoft)

macos (Macintosh), used for Apple's personal computers and work stations (MacBook, iMac).
iOS (Apple's OS for iPhone, iPad, and iPod Touch)

Android (Google's Operating System for smartphones, tablets / smartwatches).

Linux / Ubuntu (open source).



* OS acts as an intermediary between the user of computer and computer hardware, the purpose of OS is to provide an environment in which user performs functions or runs program conveniently and efficiently.

⇒ Functions of OS:-

① Memory Management:-

- (i) Keeps track of primary memory, i.e., what part of it are in use by whom, what part are not in use.
- (ii) In multiprogramming, the OS decides which process will get memory when and how much.
- (iii) Allocates the memory when a process requests it to do so.
- (iv) De-allocates the memory when a process no longer needs it or has been terminated. (Allocate and free memory)

② Processes management:-

- (i) Keeps track of processor and status of process. The program responsible for this task is known as traffic controller.
- (ii) Allocates the processor (CPU) to a process.
- (iii) De-allocates processor when a process is no longer requested. (End, abort, Load, Execute, Queue process, terminate process).

③ Resource Management:-

- (i) Keeps track of all devices. Program responsible for this task is known as the I/O controller.
- (ii) Decides which process gets the device when and for how much time.
- (iii) Allocates the device in the efficient way.
- (iv) De-allocates device. (Request device, release device, Read, write, Get device attributes, set device attributes).

④ Storage / File Management:-

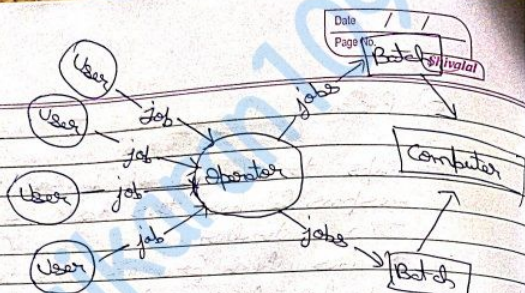
- (i) Keeps track of information, location, uses, status etc. The collective facilities are often known as file system.
- (ii) Decides who gets the resources.
- (iii) Allocates the resources.
- (iv) De-allocates the resources. (Create file, Delete file, Open, close, Read, write, Get file attributes, Set file attributes).
- (v) Security and Privacy.

⇒ Types of Operating Systems:-

- ① Batch OS:- In batch OS same type of processes are batched together for execution. E.g. Bank Statements, Payroll System.

⇒ Problem with Batch OS

- (i) Starvation
- (ii) Not Interactive



(i) Starvation - No efficient working of processes. A condition where a low-priority process is indefinitely denied access to a system resource (like the CPU) because high-priority processes continually use or request that resource).

(ii) Not Interactive - No interaction due to sequential execution of programs.
 → Because of both problems, execution of program becomes slow in batch OS.

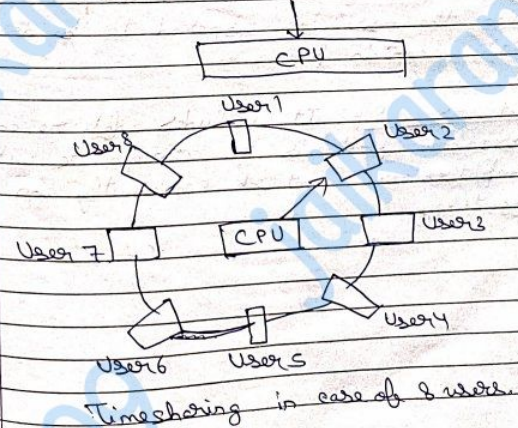
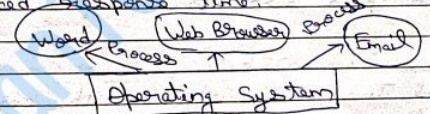
② Multiprogramming OS :

- (i) It is an extension to batch processing where the CPU is always kept busy.
- (ii) Each process needs two types of system time - CPU time and IO time.
- (iii) When a process does its I/O, the CPU can start the execution of other processes.
 - Pipeline execution
 - Non-Preemptive
- (iv) Improves the efficiency of the system.
- (v) Improves CPU utilisation.

(vi) Multiprogramming increases CPU utilization by organising jobs so that the CPU always has something to execute.

③ Multitasking / Time-Sharing OS :

- (i) It is a logical extension of a multiprogramming system.
- (ii) Enables execution of multiple programs simultaneously.
- (iii) Switching between the program.
- (iv) Preemptive.
- (v) It allows a user to perform more than one computer task at the same time.
- (vi) Reduced response time.



④ Multiprocessing OS:-

- (i) More than one processors present in the system.
 - (ii) Parallel computing is achieved.
 - (iii) Increased throughput.
 - (iv) Increased reliability.
- ⇒ Complex and sophisticated as it takes care of multiple CPUs simultaneously.

Two types;

- ① Symmetric Multiprocessing
- ② Asymmetric Multiprocessing

⑤ Distributed OS:-

The processor communicates with one another through various communications line such as high speed buses and telephone lines.

⑥ Real Time OS:-

Real Time OS exists in the case of military applications, if you want to drop a missile, then the missile is supposed to be dropped with a certain precision.

⑦ Network OS:- It includes software and associated protocols to communicate with other computers via a network conveniently and cost-effectively.

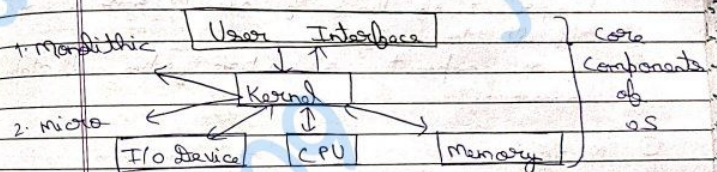
⇒ Kernel:- ① It is specific module of OS

② It is the brain of OS.

③ Gives set of instructions to hardware. There are its 2 types;

- (i) Monolithic Kernel
- (ii) Micro Kernel

Windows OS Kernel → NT



⇒ Kernel Functions:-

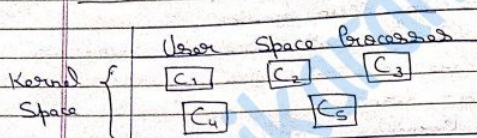
- ① Disk Management
- ② Task Management or CPU Scheduling
- ③ Memory Management
- ④ IPC (Inter Process Communication)

(Access by User → User Space)
(Not Access by User → Kernel Space)

⇒ User Mode v/s Kernel Mode:-



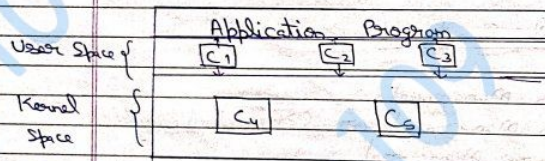
① Monolithic Kernel:



Characteristics:

- ① Faster
 - ② Larger no. of lines of code (larger in size)
 - ③ Difficult to modify
 - ④ If one component crash, entire system will stop.
- Eg: Windows NT

② Micro Kernel:



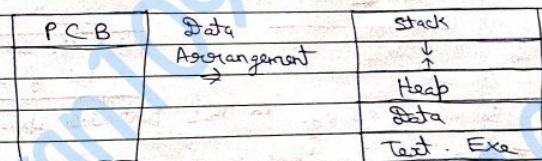
Characteristics:

- ① Smaller in size
 - ② Slow
 - ③ Less no. of lines of code
 - ④ Easy to modify (or) debugging
 - ⑤ Easy to add new functionality
 - ⑥ If one component crash, other keep running.
- Eg: Linux, Mac OS etc.

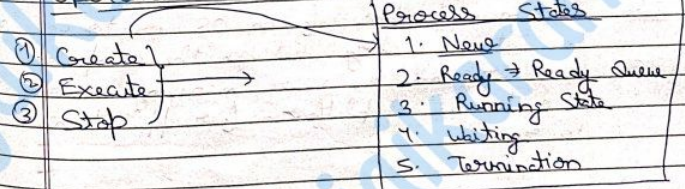
⇒ Process: Program in its executable state is known as process.
PCB ⇒ Process control Block / Process

⇒ Process Attributes:

- ① Process name ($P_1, P_2, P_3, \dots, P_n$)
- ② Process id
- ③ Process state
- ④ Process priority
- ⑤ General Purpose Register
- ⑥ Files accessed by processes
↳ Resources to read and write the data while executing
- ⑦ Program Counter
(Details of sequence, according to which program will execute)



⇒ Operations on Process:



⇒ 7 - State Process (State Transition Diagram):

Create Process

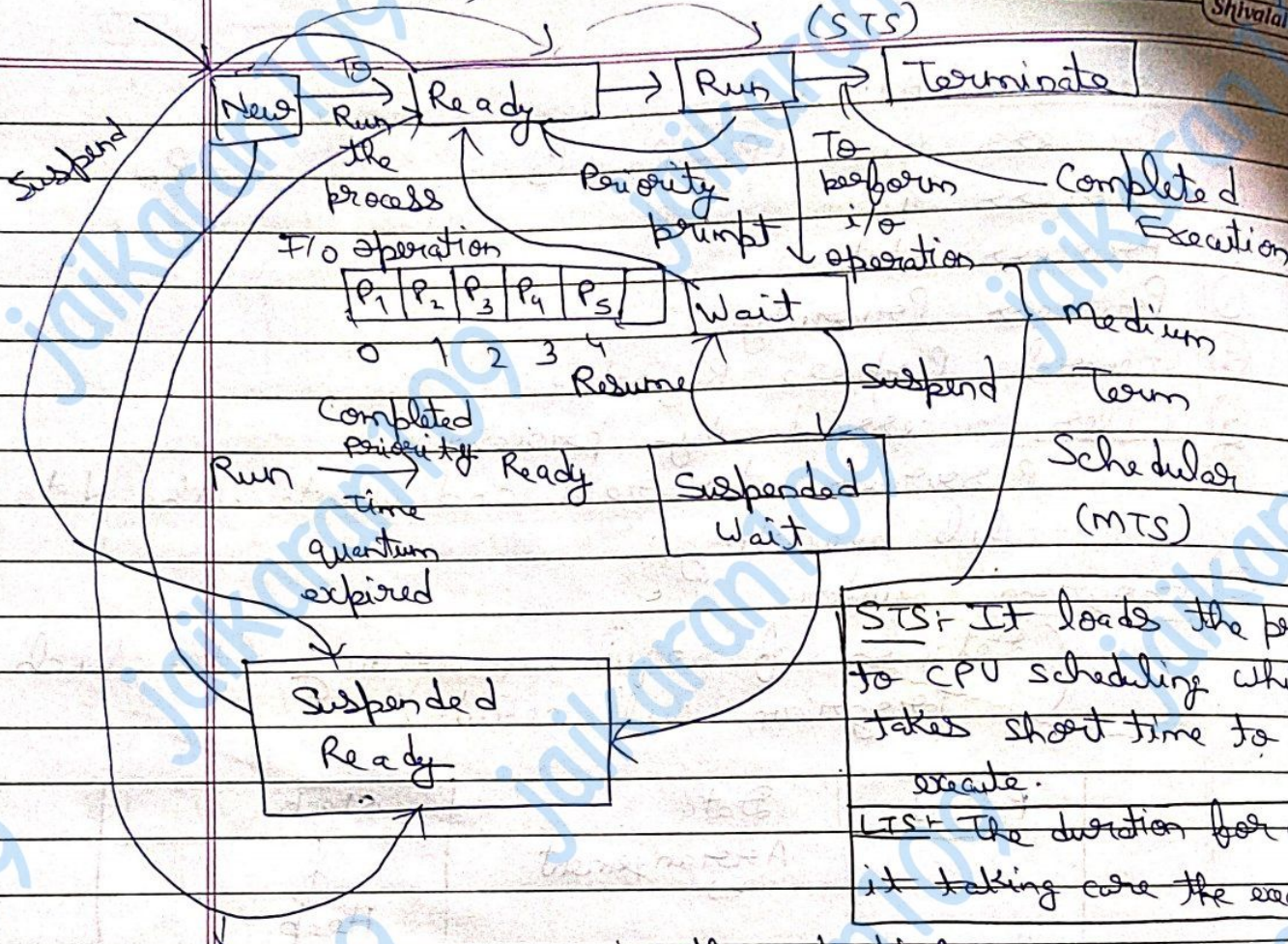
(LTS)

To start execution

Date / /

Page No.

Shivalal



Secondary memory in this particular space.

⇒ Some Questions :-

Q+1 (i) What's the main purpose of an OS?

Soln The main purpose of an OS is to act as an intermediary between the computer hardware and the user, managing resources and provide a platform for applications to run.

(ii) State different types of OS?

- Soln
- ① Batch OS
 - ② Multiprogramming OS
 - ③ Multitasking OS
 - ④ Multiprocessing OS
 - ⑤ Distributed OS
 - ⑥ Real Time OS
 - ⑦ Networked OS

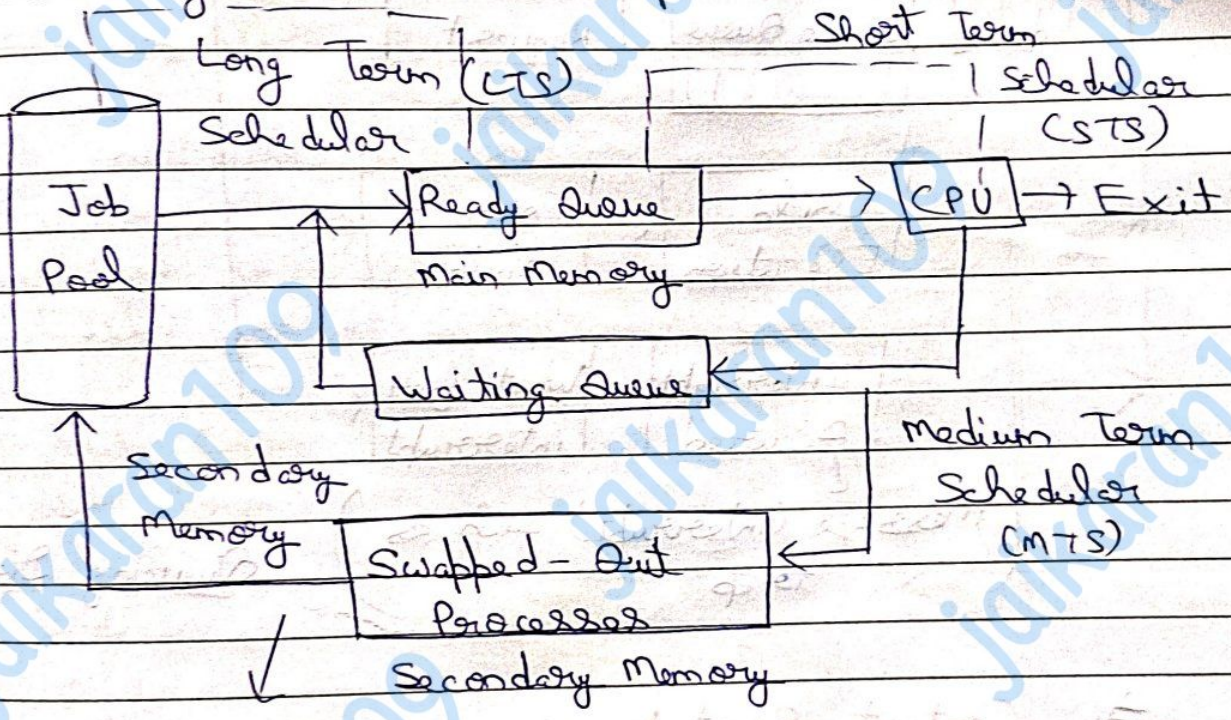
→ Scheduler :-

← (In Secondary memory)

① Long Term Scheduler (LTS) :- Job Scheduler
(Pick up the processes from job pool)

② Short Term Scheduler (STS) :- CPU Scheduler
(In main memory)

③ Medium Term Scheduler (MTS) :- Swapping in and swapping out.
(Waiting State to suspended state (swap out))



Scheduler's Diagram (In which sequence, we have to insert processes).

⇒ Process Queue :- (Queuing diagram)

In order to understand how process transfer from one queue to another queue during execution.
(Context Switching diagram).

Process	CT	TAT	WT
P ₁	5	5	5-5 = 0
P ₂	8	7	7-3 = 4
P ₃	16	14	14-8 = 6

⇒ Non-Preemptive :- If one process is executing, then no other process arrives in the processor, till the first process completed.

⇒ Turn Around Time = (Completion - Arrival) Time
(TAT) Time
Time b/w arrival and completion of process in a system.

⇒ Waiting Time (WT) :- (TAT - Execution Time)
(How much time it waits to load in processor for execution).

⇒ Average TAT :-

$$= \frac{\text{Total TAT}}{\text{No. of processes executed}}$$

$$= \frac{5+7+14}{3} = \frac{26}{3} = 8.6 \text{ ms}$$

⇒ Average WT :-
$$= \frac{\text{Total WT}}{\text{No. of processes executed}}$$

$$= \frac{0+4+6}{3} = \frac{10}{3} = 3.3 \text{ ms}$$

There is no significance of response time in this case because it is non-preemptive mode of algorithm.

⇒ Response Time = Starting Time - Arrival Time

$$P_1 = 0 - 0 = 0 \text{ ms}$$

$$P_2 = 5 - 1 = 4 \text{ ms}$$

$$P_3 = 8 - 2 = 6 \text{ ms}$$

⇒ Waiting time is equal to response time in case of non-preemptive scheduling algorithm (mode of execution) (Execution)

Process	A.T (in msec)	Execution Time (E.T) (in msec)
P ₁	0	4
P ₂	1	3
P ₃	2	1
P ₄	3	2
P ₅	4	5

Use FCFS Scheduling algorithm

(i) Draw Gantt chart

(ii) Calculate Avg. TAT

(iii) Calculate Avg. WT



(i) Average TAT = $\frac{\text{Total TAT}}{\text{No. of processes executed}}$
 $= \frac{34}{5} = 6.8 \text{ ms}$

(ii) Average WT = $\frac{\text{Total W.T}}{\text{No. of processes executed}}$
 $= \frac{19}{5} = 3.8 \text{ ms}$

② Shortest Job First (SJF) -

Non Preemptive mode of process

Criteria - Execution Time of a process

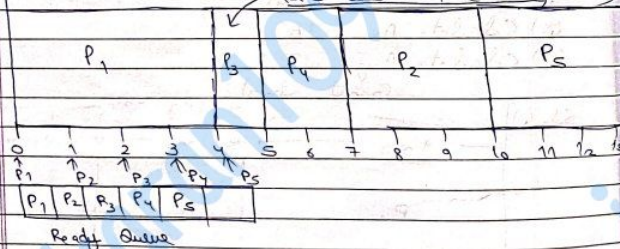
According to this algorithm

shortest process (small execution time) will be selected

(Selection according to increasing E.T)

Based on previous ques

Gantt Chart -



Process	A.T	E.T	C.T	TAT	W.T
P ₁	0	4	4	4	0
P ₂	1	3	6	9	6
P ₃	2	1	5	3	2
P ₄	3	2	7	4	2
P ₅	4	5	15	11	6

Avg. TAT = 6.2ms, Avg. WT = 3.2ms

Avg. TAT Avg. WT

FCFS	6.8ms	3.8ms
SJF	6.2ms	3.2ms

(SJF) Takes less time in Turn around and waiting than FCFS.

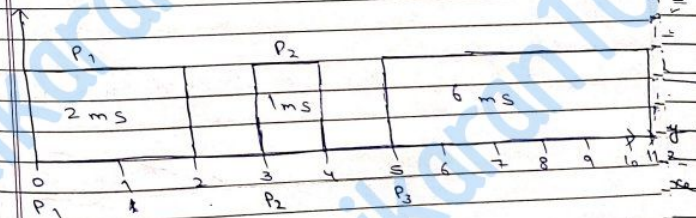
Process	A.T	E.T
P ₁	0	2
P ₂	3	1
P ₃	5	6

Use SJF scheduling algorithm

(i) Draw Gantt chart

(ii) Calculate Avg TAT and avg. WT

Sol: (i) Gantt chart



Process	A.T	E.T	TAT	W.T	C.T
P ₁	0	2	2	0	2
P ₂	3	1	1	0	4
P ₃	5	6	6	0	11

Avg. TAT = $\frac{9}{3} = 3 \text{ msec}$

Avg. WT = 0 msec.

* When a long process blocks small process to complete its execution (generally for P_1 in SJF) is called conveying.

Process	A.T	E.T
P_1	0	24
P_2	1	3
P_3	2	3

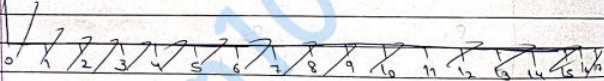
Use SJF Scheduling;

(i) Draw Gantt Chart.

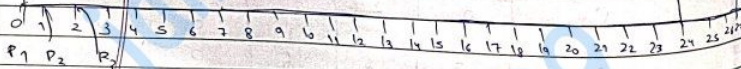
(ii) Calculate Avg. TAT.

(iii) Calculate Avg. WT.

Sol. (i) Gantt Chart



24 ms



	A.T	E.T	C.T	(C.T - A.T) TAT	(E.T - A.T) WT
P_1	0	24	24	24	0
P_2	1	3	24	26	23
P_3	2	3	28	28	25

(vi) Avg. TAT = $\frac{\text{Total TAT}}{\text{No. of processes executed}}$

$$= \frac{78}{3} = 26 \text{ msec}$$

(vii) Avg. WT = $\frac{\text{Total WT}}{\text{No. of processes executed}}$

$$= \frac{48}{3} = 16 \text{ msec}$$

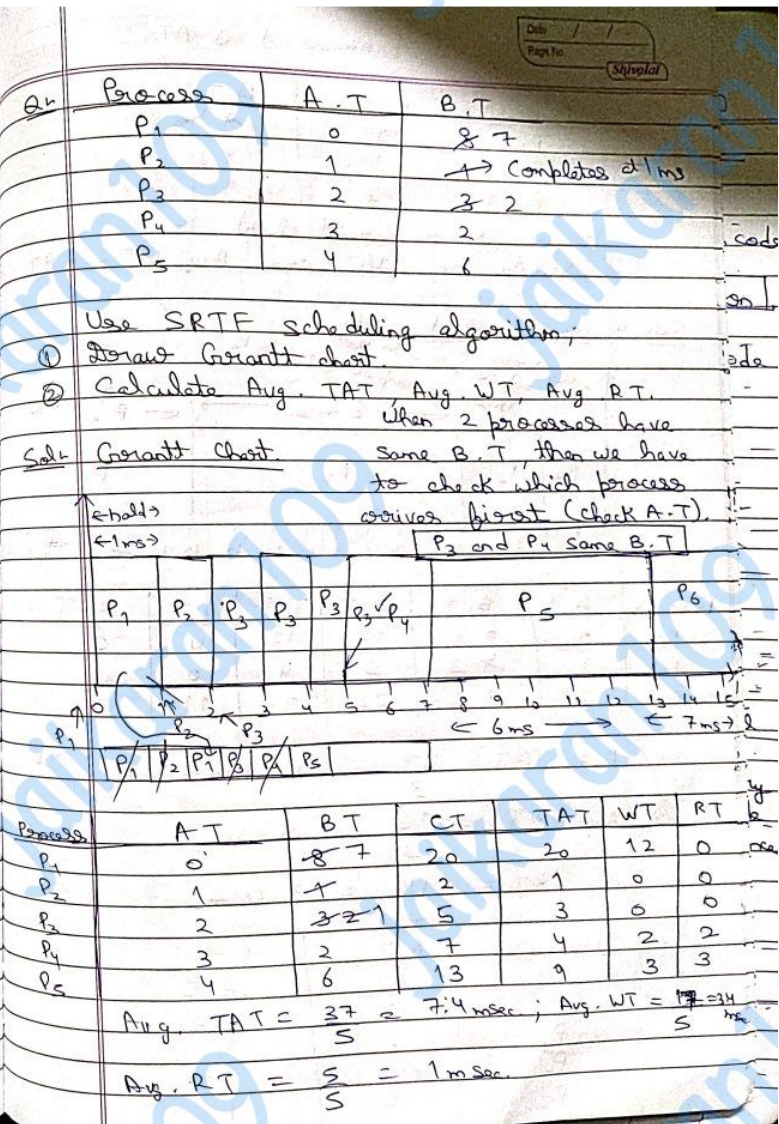
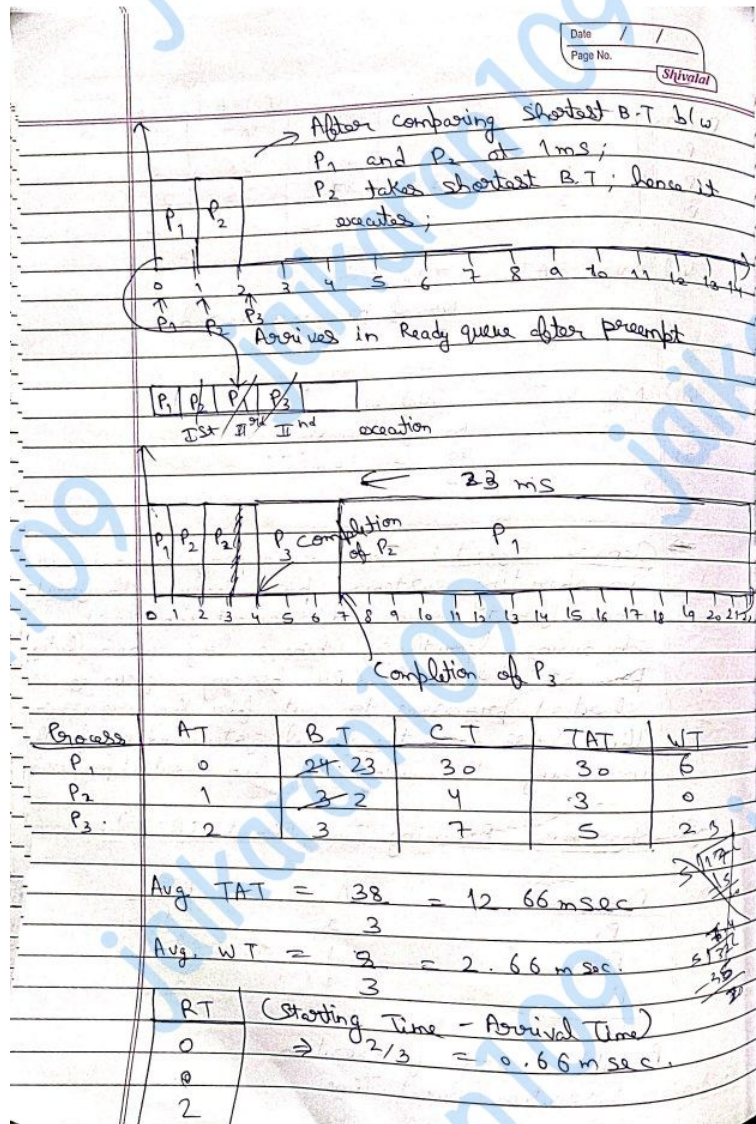
Convey Effect: This is a performance issue in operating systems, particularly with first come first serve and shortest job first scheduling algorithms, where a long running process holds the CPU, forcing a queue of shortest processes to wait for the processor. This slows down the overall system and significantly increasing the waiting time for shortest processes.

③ Shortest Remaining Time First (SRTF):

Process	AT	BT
P_1	0	24
P_2	1	3
P_3	2	3

Premptive mode of Execution

Wait Execution / Burst Time



Practice
Ques If arrival time is not given; assume all processes at 0 AT.

Process	A.T	B.T
P ₁	0	9
P ₂	1	4
P ₃	2	9

- (i) Draw SRTF Gantt chart.
(ii) Calculate Avg. TAT and Avg. WT and RT.

Soln (i) Gantt chart

Process	A.T	B.T	C.T	TAT	W.T	R.T
P ₁	0	9	13	13	4	0
P ₂	1	4	5	4	0	0
P ₃	2	9	22	20	11	11

$$\text{Avg. TAT} = \frac{37}{3} = 12.33 \text{ msec}$$

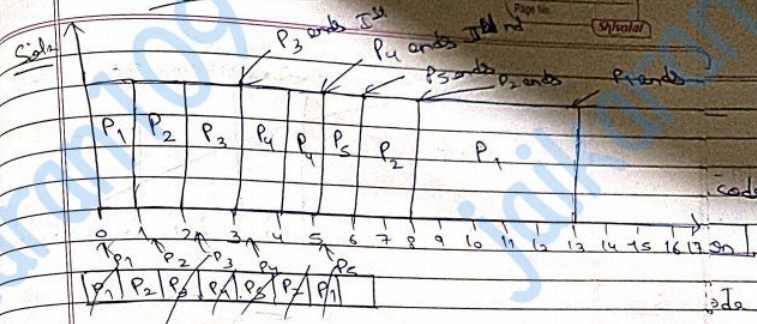
$$\text{Avg. WT} = \frac{15}{3} = 5 \text{ msec}$$

$$\text{Avg. RT} = \frac{11}{3} = 3.66 \text{ msec}$$

Q2 Process

Process	A.T	ET
P ₁	0	6
P ₂	1	3
P ₃	2	1
P ₄	3	2
P ₅	5	1

By SRTF



Process	A.T	ET	C.T	TAT	WT	RT
P ₁	0	6	13	13	7	0
P ₂	1	3	8	7	4	0
P ₃	2	1	3	1	0	0
P ₄	3	2	5	2	0	0
P ₅	5	1	6	1	0	0

$$\text{Avg. TAT} = \frac{24}{5} = 4.8 \text{ msec}$$

$$\text{Avg. WT} = \frac{11}{5} = 2.2 \text{ msec}$$

$$\text{Avg. RT} = 0 \text{ msec}$$

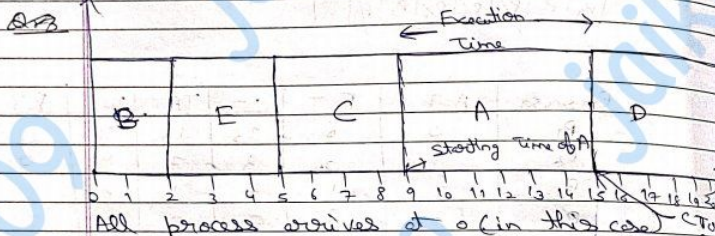
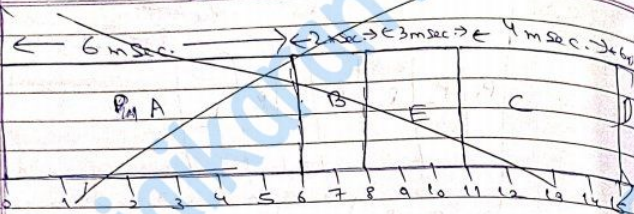
Q2

Process	A.T	ET
A	0	6
B	0	2
C	0	4
D	0	6
E	0	3

Using SJF (Calculate the avg TAT)

We will check process id
if AT and ET both are
Same.

Date: / /
Page No. Shivani



All process arrives at 0 (in this case)

Process	AT	ET	CT	TAT	WT	RT
A	0	6	15	15	9	9
B	0	2	2	2	0	0
C	0	4	9	9	5	5
D	0	6	21	21	15	15
E	0	3	5	5	2	2

$$\text{Avg. TAT} = \frac{52}{5} = 10.4 \text{ msec}$$

$$\text{Avg. WT} = \frac{31}{5} = 6.2 \text{ msec}$$

Because it is non-preemptive process,
WT = RT

⇒ SRTF is also known as Shortest Job
First (SJF) in preemptive mode.

Priority Scheduling Algorithm

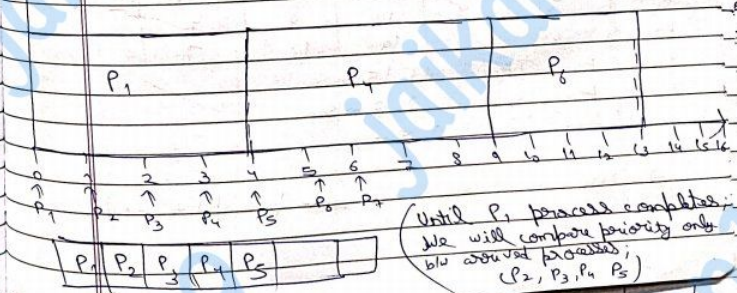
1. Mode:
 - Non Preemptive
 - Preemptive

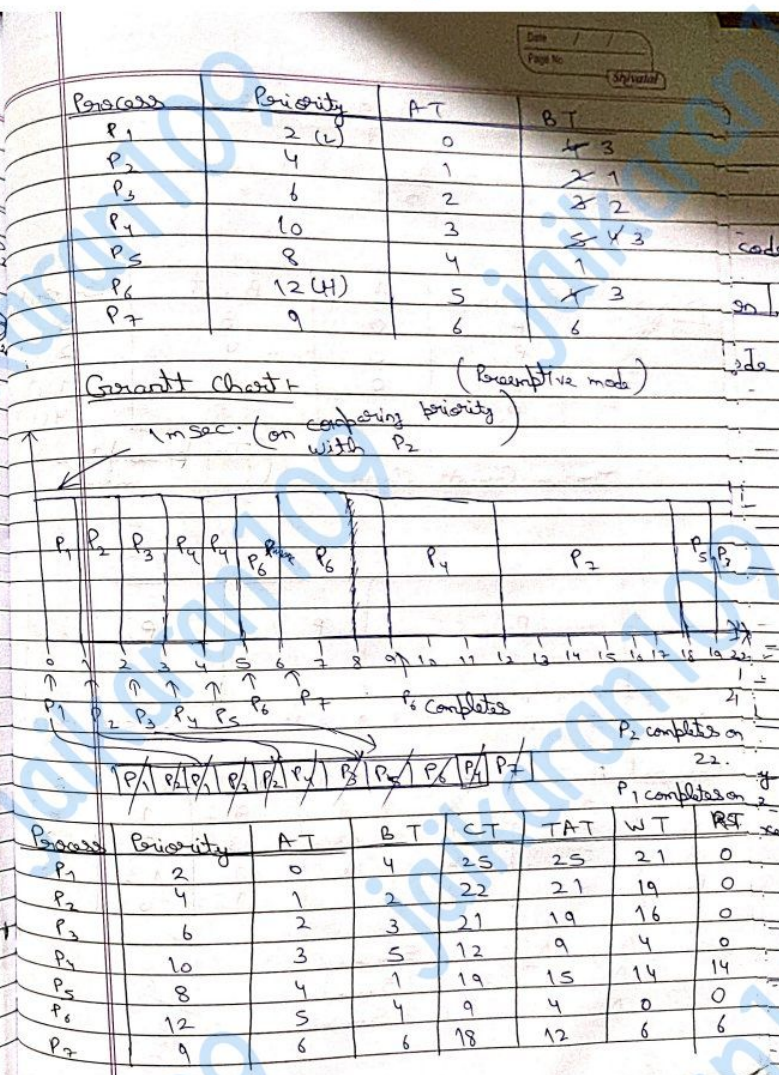
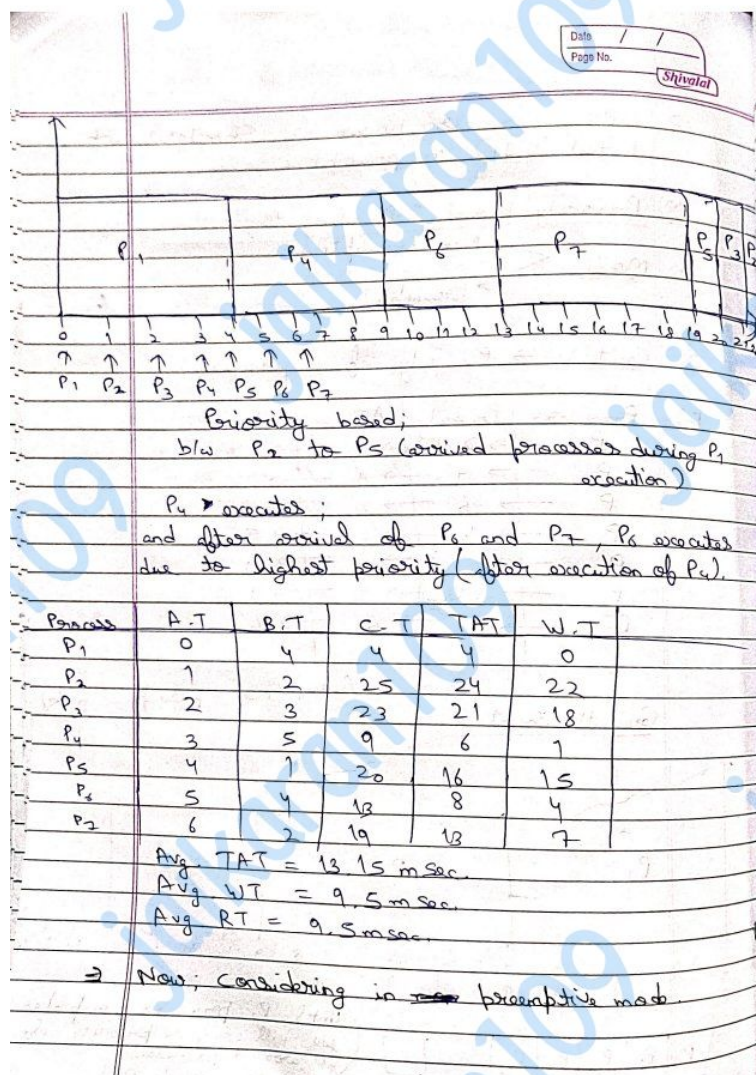
2. Criteria: Priority of a process

Ex. Process	Priority	AT	BT/ET
P ₁	→ 2 (Low)	→ 0	→ 4
P ₂	→ 4	→ 1	→ 2
P ₃	→ 6	→ 2	→ 3
P ₄	→ 10	→ 3	→ 5
P ₅	→ 8	→ 4	→ 1
P ₆	→ 12 (High)	→ 5	→ 4
P ₇	→ 9	→ 6	→ 6

(In non-preemptive mode)

Grant Chart



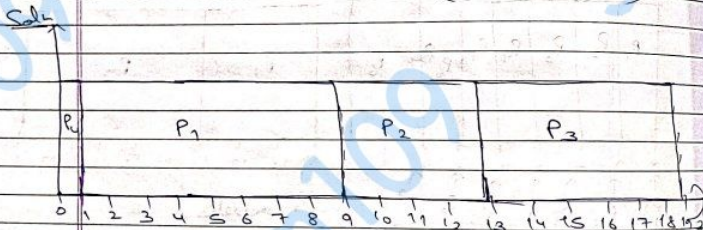


$$\text{Avg. TAT} = \frac{105}{7} = 15 \text{ msec.}$$

$$\text{Avg. WT} = \frac{80}{7} = 11.2 \text{ msec.}$$

Process	A.T	B.T	Priority
P ₁	0	8	3
P ₂	0	4	2
P ₃	0	6	1 (W)
P ₄	0	1	4 (H)

(i) Draw Gantt chart. (In preemptive mode.)



Process	A.T	B.T	C.T	TAT	WT	R.T
P ₁	0	8	9	9	1	1
P ₂	0	4	13	13	9	9
P ₃	0	6	19	19	13	13
P ₄	0	1	1	1	0	0

$$\text{Avg. TAT} = 10.5 \text{ msec.}$$

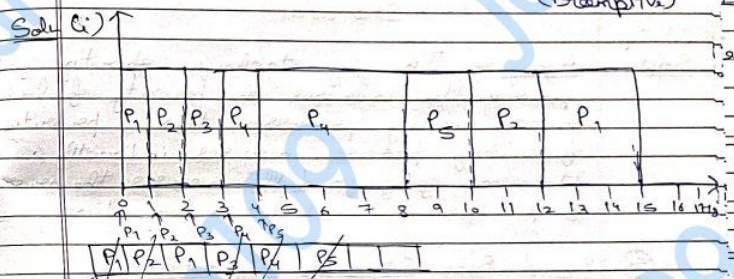
$$\text{Avg. WT} = 5.75 \text{ msec.}$$

$$\text{Avg. RT} = 5.75 \text{ msec.}$$

(Priority)

Process	A.T	B.T	Priority
P ₁	0	4	3
P ₂	1	3	2
P ₃	2	1	1
P ₄	3	5	4
P ₅	4	2	5

(Preemptive)



Process	A.T	B.T	C.T	TAT	WT	RT
P ₁	0	4	15	15	11	0
P ₂	1	3	12	11	8	0
P ₃	2	1	3	1	0	0
P ₄	3	5	8	5	0	0
P ₅	4	2	10	6	4	4

$$\Rightarrow \text{Avg. TAT} = \frac{38}{5} = 7.6 \text{ msec.}$$

$$\Rightarrow \text{Avg. WT} = \frac{23}{5} = 4.6 \text{ msec.}$$

$$\Rightarrow \text{Avg. RT} = \frac{4}{5} = 0.8 \text{ msec.}$$

(Preemptive)

→ Disadvantage of Priority Scheduling

A major problem with priority scheduling algorithm is indefinite blocking of lower priority processes, which creates a situation of starvation for a lowest priority process.
 ↳ Starvation

Solution: Ageing ⇒ In ageing technique, operating system periodically increases the priority of lowest priority waiting processes, thus, a priority of waiting process increases indirectly. It means ageing gradually increases the priority of process that wait for a long time.

⑤ Round Robin Scheduling Algorithm:

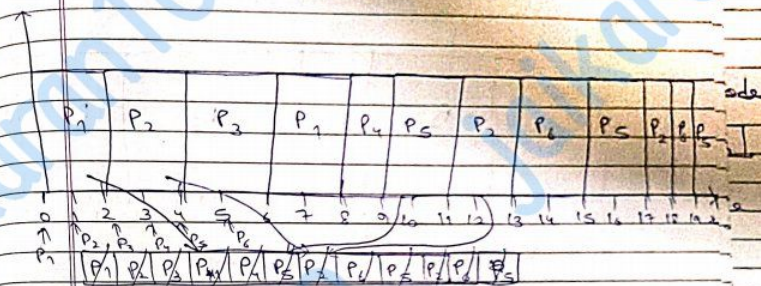
Mode: Preemptive (By default)

Criterion: Arrival Time

Process	A.T	B.T
P ₁	0	4
P ₂	1	5
P ₃	2	2
P ₄	3	1
P ₅	4	6
P ₆	5	3

Time Quantum = 2 msec

Sol: Gantt Chart:



CT	TAT	WT	RT
8	8	4	0
18	17	12	1
6	4	2	2
9	6	5	5
21	17	11	5
19	14	11	8

$$\rightarrow \text{Avg TAT} = \frac{66}{6} = 11 \text{ msec}$$

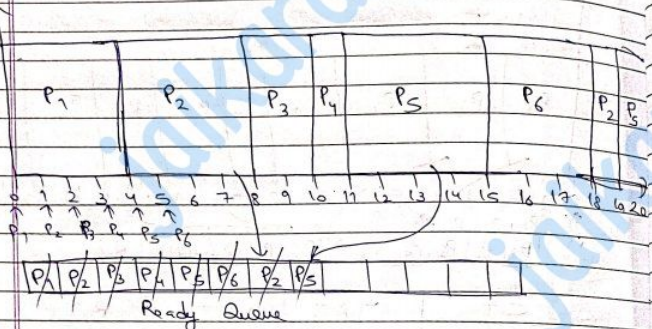
$$\text{Avg WT} = \frac{45}{6} = 7.5 \text{ msec}$$

$$\text{Avg RT} = 3.5 \text{ msec}$$

Process	A.T	B.T
P ₁	0	X ✓
P ₂	1	S 1
P ₃	2	X ✓
P ₄	3	X ✓
P ₅	4	X 2
P ₆	5	3 -

Time Quantum = 4 msec

Soln



Process	A.T	B.T	C.T	TAT	W.T	R.T
P ₁	0	4	4	4	0	0
P ₂	1	5	19	18	13	3
P ₃	2	2	10	8	6	6
P ₄	3	1	11	8	7	7
P ₅	4	6	21	17	11	7
P ₆	5	3	18	13	10	10

Avg. TAT = $\frac{68}{6} = 10.13 \text{ msec}$

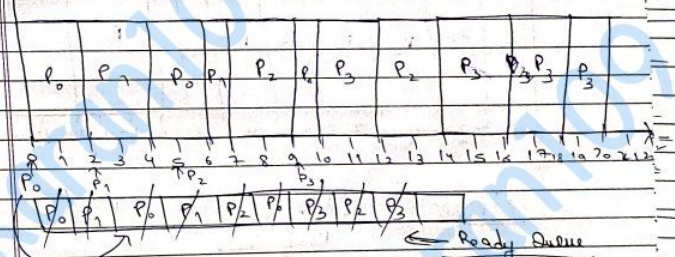
Avg. WT = $\frac{47}{6} = 7.83 \text{ msec}$

Avg. RT = $\frac{33}{6} = 5.5 \text{ msec}$

Qr The following snapshots of the processes are mentioned with no I/O activities. Calculate Turnaround Time, Waiting Time and Response Time for all processes for Round Robin CPU scheduling algorithm. Time slice = 2ms

Process	A.T	B.T
P ₀	0	5
P ₁	2	3
P ₂	5	4
P ₃	9	8

Soln



Process	A.T	B.T	C.T	TAT	W.T	R.T
P ₀	0	5	10	10	5	0
P ₁	2	3	7	5	2	0
P ₂	5	4	14	9	5	2
P ₃	9	8	20	11	3	1

Avg. TAT = 8.75 msec

Avg. WT = 3.75 msec

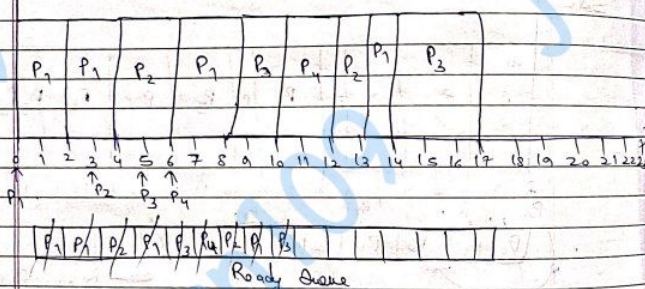
Avg. RT = 0.75 msec

Q1 (2024)

Q1 Calculate TAT, WT and RT of following four processes
Context switching: when any one process present and another process arrives in processor.

Process	AT	BT	Time Slice
P ₁	0	7	1
P ₂	3	3	1
P ₃	5	5	3
P ₄	6	2	1

Soln



Process	AT	BT	CT	TAT	WT	RT
P ₁	0	7	14	14	7	0
P ₂	3	3	13	10	7	1
P ₃	5	5	17	12	7	3
P ₄	6	2	12	6	0	4

Avg. TAT = 10.5 msec

Avg. WT = 6.25 msec

Avg. RT = 2 msec

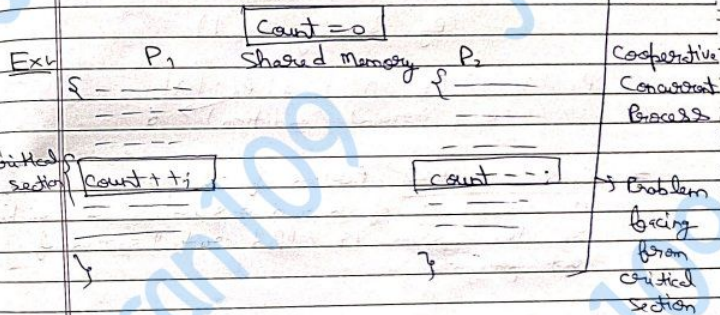
⇒ Inter Process Communication

Process → Independent

→ Cooperative Processes (Concurrent)

(Execution on same time)

Process Synchronization



- ① $\text{mov } R_0, \text{count}$ (Source: count, Destination: R₀)
- ② $\text{INR } R_0$
- ③ $\text{mov count}, R_0$
- ④ $\text{DEC } R_1$
- ⑤ $\text{mov count}, R_1$

⇒ The portion of program in which the data stored in register modifies is called critical section.
(By First Come First Serve)

- P₁: 1, 2, 3 P₂: 1, 2, 3
- ① R₀ = 6
 - ② R₁ = 6-1
 - ③ count = R₁ = 5

⇒ Solution to Critical Section Problem

→ Synchronisation Mechanism → Peterson Algorithm

It is used in case of only two cooperative concurrent processes.

→ Semaphore → It is used in case of more than two (or) two cooperative processes.

⇒ Peterson Algorithm (Final)

boolean flag[2]; → Indicate which process wants to enter in its critical section.

int turn; → Indicate whose turn it is to enter in a critical section.

Process P_i
do {
while (flag[j] == true && turn == j);
[Critical Section]

Enter Section Code
Exit Section Code
flag[i] = false;
Remainder Section
while (1)

Process P_j

do {
[Entry section]
while (flag[i] == true && turn == i);
[Critical section]
flag[j] = false;
Remainder section;
[Exit section]
while (1)

⇒ Entry Section of Code: Wait, down, P(), another name of Semaphore entry section code.

⇒ Exit Section of Code: Signal, Up, V() (Semaphore)

⇒ Semaphore (S): It is an integer variable.

Process P_i

do {
wait(S); → Entry section code
[Critical Section]
signal(S); → Exit section code
Remainder section;
while (1)

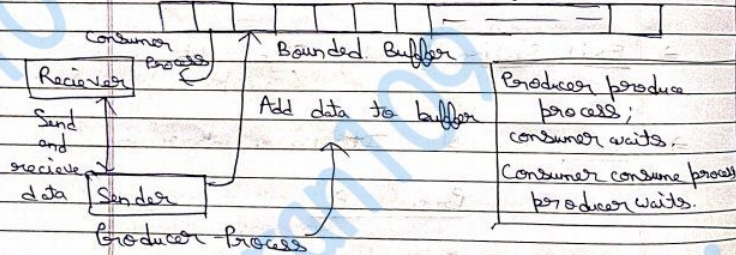
int s = 4

Ex. wait(s); $\rightarrow 4-1=3$
wait(s); $3-1=2$
wait(s); $2-1=1$
signal(s); $1+1=2$
signal(s); $2+1=3$

⇒ There are its two types; mutex

- ① Binary Semaphore $\Rightarrow S = 0, 1$
- ② Counting Semaphore $\Rightarrow S = -\infty, +\infty$

⇒ Producer-Consumer Problem (or) Bounded Buffer Problem
Consume data from buffer



Problem - We have buffer of fixed size. A producer process can produce an item and place in the buffer.
Statement - A consumer process can pick items from buffer and consume them.
We need to ensure that when a producer is placing an item in the buffer, then at the same time, consumer should not consume any item process

from the buffer.
In this problem, buffer is the critical section

Solution by using Semaphore

Buffer empty; mutex = 1; (no overflow)
Full = 0;
Empty = n; (n no. of slots are empty)

Full = n; Buffer (overflow)
empty = 0; full

Producer

do {
f {
wait(empty); // To check for overflow
wait(mutex);
[Critical Section] Add data in buffer
Signal(mutex);
Signal(full);
}
}
Consumer {
while (full == 0) → Underflow
empty = n

do {
wait(full); // To check for underflow
wait(mutex);
[Critical Section]
Signal(mutex);
Signal(empty);
while (1)
}
}

⇒ ^(In entry section)
 sem wait(mutex)
 $\{ \text{while (mutex} \leq 0);$
 $\text{mutex} = \text{mutex} - 1;$
Critical section
 $\}$
 → consume an item from buffer

⇒ $\text{signal(mutex);} \rightarrow \{$
 $\text{mutex} = \text{mutex} + 1;$
 $\}$

⇒ $\text{signal(empty)} \{ \text{empty} = \text{empty} + 1;$
 $\}$

⇒ Reader-Writer Problem:-

Read	→	Read	✓	(Allow)
Read	→	Write	✗	[Block]
Write	→	Read	✗	
Write	→	Write	✗	

⇒ Solution of Reader Writer Problem:-

Semaphore = sem
 int read count

For ~~writer~~ ^{writer}
 do {
 wait(sem);
write operation performed
 signal(sem);
 $\}$
 $\text{while}(1);$

For reader process
 do {
 wait(mutex);
 $\text{read count}++;$
 $\text{if (read count} == 1)$
 wait(sem);
 signal(mutex);
Perform Read operation
 wait(mutex);
 $\text{read count}--;$
 $\text{if (read count} == 0)$
 signal(sem);
 $\}$